lab5 实验报告

PB24111621 肖瀚

一、实验目的与内容

- 实验目的：将上一次实验设计的单周期 CPU 流水化，形成一个不考虑冒险的流水线 CPU。
- 实验内容：修改寄存器堆，实现写优先，并完成对应的思考题目；添加段间寄存器，具备 flush 功能，根据数据通路连接好各个端口信号，通过仿真框架验证和 FPGA 上板测试。

二、逻辑设计

任务一：写优先的寄存器堆

只需要在原有的寄存器堆的基础上修改读的逻辑，如果写使能有效并且读地址和写地址一致(非 0 号寄存器)，那么就会将写的数据直接读出来，传给相应的 `rf_rd` 。代码实现如下：

```
module REG_FILE (
    input                   [ 0 : 0]        clk,

    input                   [ 4 : 0]        rf_ra0,
    input                   [ 4 : 0]        rf_ra1,
    input                   [ 4 : 0]        rf_wa,
    input                   [ 0 : 0]        rf_we,
    input                   [31 : 0]        rf_wd,
    input                   [ 4:0]          debug_reg_ra,

    output                  [31 : 0]        rf_rd0,
    output                  [31 : 0]        rf_rd1,
    output                  [31 : 0]        debug_reg_rd
);

    reg [31 : 0] reg_file [0 : 31];

    // 用于初始化寄存器
    integer i;
    initial begin
        for (i = 0; i < 32; i = i + 1)
            reg_file[i] = 0;
    end

    always @(posedge clk) begin
        if(rf_we && rf_wa!=0)
            reg_file[rf_wa] <= rf_wd;
    end

    assign rf_rd0 = (rf_we && rf_wa != 5'b0 && rf_ra0 == rf_wa) ? rf_wd : reg_file[rf_ra0];
    assign rf_rd1 = (rf_we && rf_wa != 5'b0 && rf_ra1 == rf_wa) ? rf_wd : reg_file[rf_ra1];
    assign debug_reg_rd = reg_file[debug_reg_ra];
endmodule
```

思考题回答

对于本次实验中的五级流水线 CPU，连续执行以下的指令序列后，若寄存器堆没有使用写优先，运行结束后 x4 与 x5 中的结果是什么?

```
addi x1, x0, 1
addi x2, x0, 2
addi x3, x0, 3
addi x1, x0, 10
addi x2, x0, 20
addi x3, x0, 30
addi x0, x0, 0
add  x4, x1, x2
add  x5, x2, x3
nop
nop
nop
```

由于 x1, x2, x3 分别在第8, 9, 10周期被写入寄存器，

`add  x4, x1, x2` 在第9周期进入 ID 阶段，故若没有写优先，那么读到的 x1, x2 分别是10和2，故 x4 = 12

`add  x5, x2, x3` 在第10周期进入 ID 阶段，若没有写优先，读到的 x2, x3 分别是 20和3，故 x5 = 23

任务二：无冒险流水线

flush信号产生

在 EX 阶段，当发现有跳转指令，并且实现了跳转，也就是**下一个 PC 不是 pcadd4**，那么此时 flush 信号就会被设置为 1，否则保持为 0。

段间寄存器的 flush 功能

以 `IF/ID` 寄存器为例， `ID/EX` 同理，先确定寄存器的端口信号，然后将对应的 if 改为 id 以便后面传输，如果复位信号 rst 有效，那么所有信号清零，否则若 flush 信号有效，则清除 commit 信号以及其他控制信号，由于本次实验说明不考虑 stall 功能，则若 flush 再无效，则输出会更新。具体实现的代码如下：

```
module IF_ID_REG(
    input         clk,
    input         rst,
    input         en,
    input         flush,      // 分支冲刷
    // 输入 IF 段
    input  [31:0] if_pc,
    input  [31:0] if_pc_add4,
    input  [31:0] if_inst,
    input         if_commit,  // IF 段产生 commit=1
    // 输出到 ID 段
    output reg [31:0] id_pc,
    output reg [31:0] id_pc_add4,
    output reg [31:0] id_inst,
    output reg       id_commit
);
    always @(posedge clk) begin
        if(rst) begin
            id_pc       <= 32'b0;
            id_pc_add4  <= 32'b0;
            id_inst     <= 32'b0;
            id_commit   <= 1'b0;
        end
        else if(en) begin
            if(flush) begin
                // 冲刷：置为 NOP，清空 commit 和所有控制
                id_pc       <= if_pc;
                id_pc_add4  <= if_pc_add4;
                id_inst     <= 32'b0;
                id_commit   <= 1'b0;
            end
            else begin
                id_pc       <= if_pc;
                id_pc_add4  <= if_pc_add4;
                id_inst     <= if_inst;
                id_commit   <= if_commit;
            end
        end
    end
endmodule
```

而对于其余两个段间寄存器，由于本次实验 PC 每三周期更新一次，故在跳转指令运行到 EX 阶段发现跳转错误时，后面指令最多运行到 ID 阶段，故只需要清除前两个寄存器，后面两个寄存器暂时不添加 flush 端口。

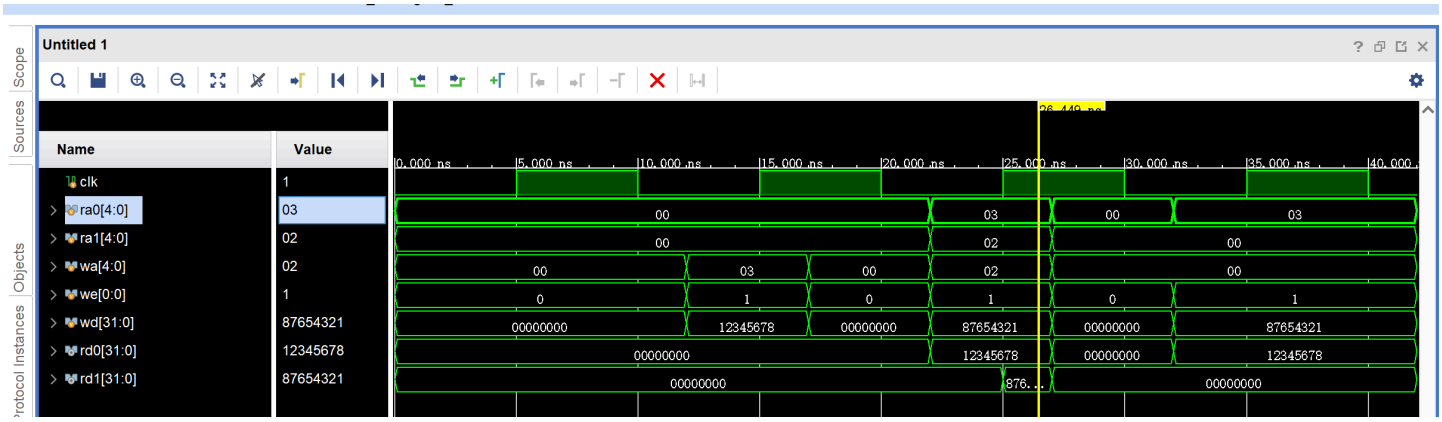
commit 信号处理

commit 信号在 IF 阶段产生，并且被设置为 1，随后的每个段间寄存器都会添加对应的 commmit 输入输出端口，这样 commit 信号便可随寄存器逐渐传递，便于后面的仿真实现。

三、仿真实现

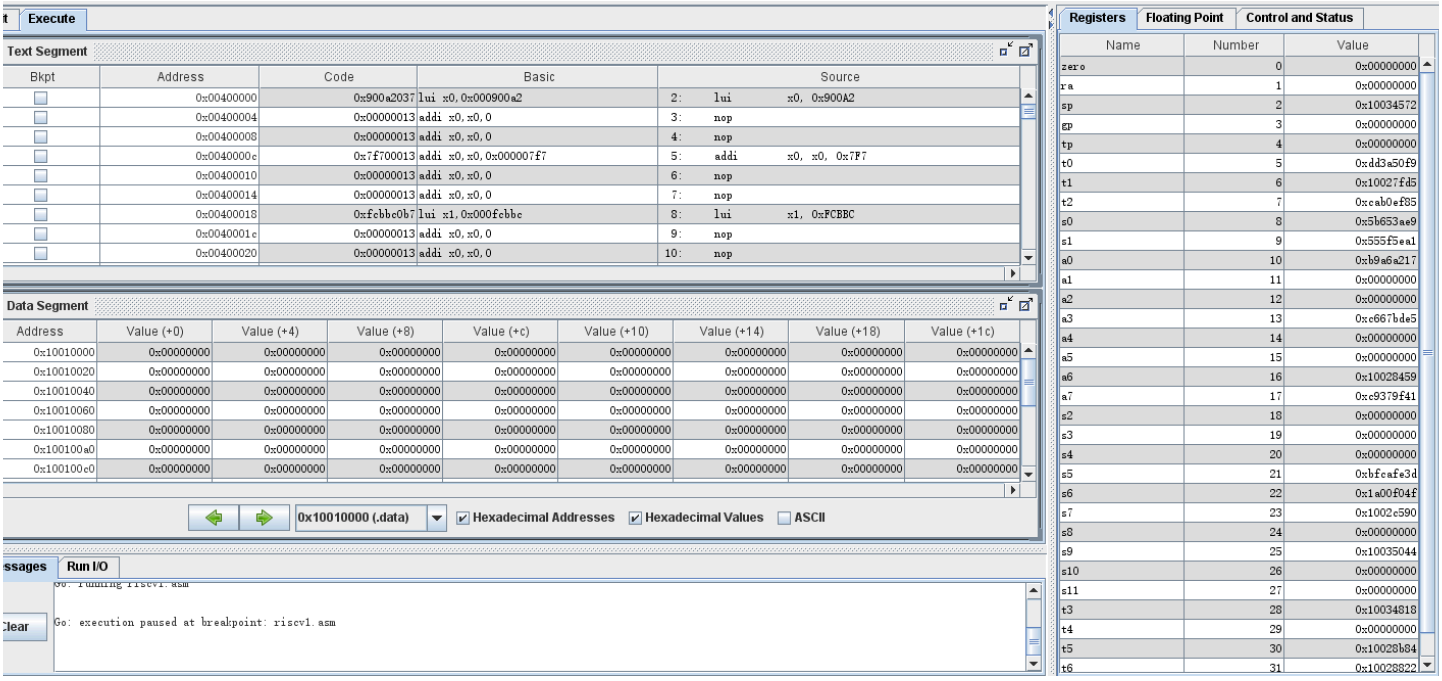
任务一

从下图可以看到，黄色部分，ra1 的地址与 写地址一致，并且写使能有效，那么读出来的值就是即将写入的值，实现了写优先。

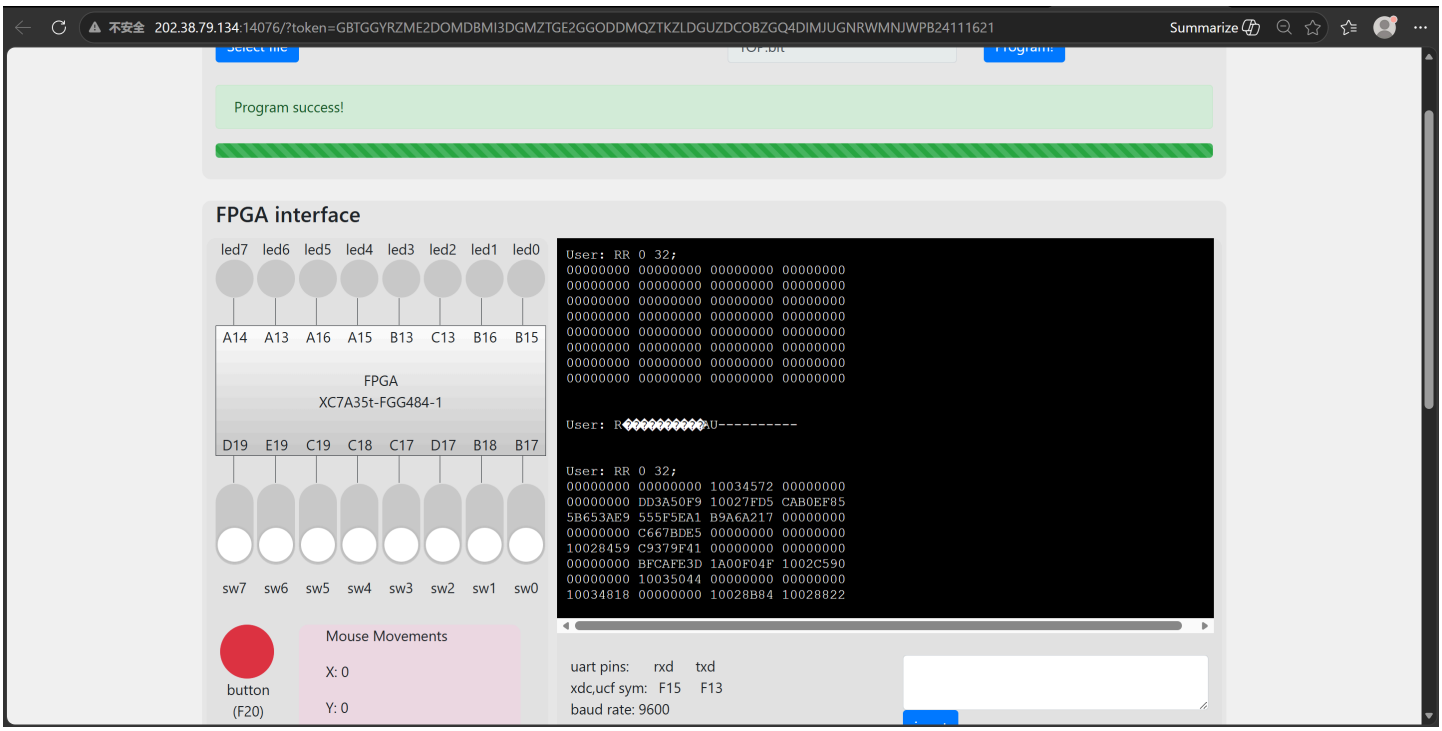


任务二

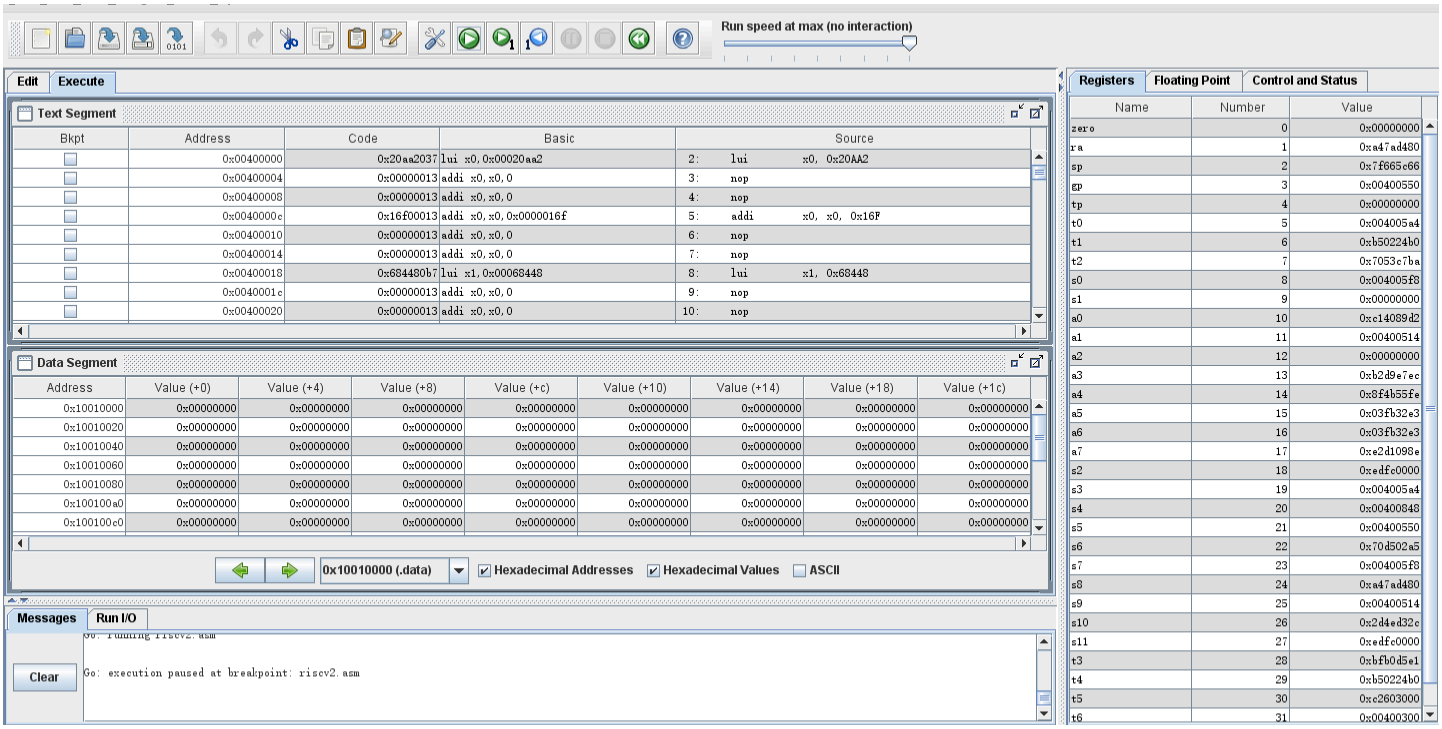
对于第一个仿真文件，RARS、仿真框架、上板运行的结果如下，三者是一致的。



```
qi@ISLA:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                           846
Instruction count:              416
PC:                             0x004006A8
Instruction:                    0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000000 [ 2] = 0x10034572 [ 3] = 0x00000000
[ 4] = 0x00000000 [ 5] = 0xDD3A50F9 [ 6] = 0x10027FD5 [ 7] = 0xCAB0EF85
[ 8] = 0x5B653AE9 [ 9] = 0x555F5EA1 [10] = 0xB9A6A217 [11] = 0x00000000
[12] = 0x00000000 [13] = 0xC667BDE5 [14] = 0x00000000 [15] = 0x00000000
[16] = 0x10028459 [17] = 0xC9379F41 [18] = 0x00000000 [19] = 0x00000000
[20] = 0x00000000 [21] = 0xBFCAFE3D [22] = 0x1A00F04F [23] = 0x1002C590
[24] = 0x00000000 [25] = 0x10035044 [26] = 0x00000000 [27] = 0x00000000
[28] = 0x10034818 [29] = 0x00000000 [30] = 0x10028B84 [31] = 0x10028822
qi@ISLA:~/COD25-Simulation-Framework$
```



对于第二个仿真文件，RARS、仿真框架、上板运行的结果如下，三者是一致的。




```
qi@ISLA:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Diffest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                           858
Instruction count:              400
PC:                             0x00400744
Instruction:                    0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0xA47AD480 [ 2] = 0x7F665C66 [ 3] = 0x00400550
[ 4] = 0x00000000 [ 5] = 0x004005A4 [ 6] = 0xB50224B0 [ 7] = 0x7053C7BA
[ 8] = 0x004005F8 [ 9] = 0x00000000 [10] = 0xC14089D2 [11] = 0x00400514
[12] = 0x00000000 [13] = 0xB2D9E7EC [14] = 0x8F4B55FE [15] = 0x03FB32E3
[16] = 0x03FB32E3 [17] = 0xE2D1098E [18] = 0xEDFC0000 [19] = 0x004005A4
[20] = 0x00400848 [21] = 0x00400550 [22] = 0x70D502A5 [23] = 0x004005F8
[24] = 0xA47AD480 [25] = 0x00400514 [26] = 0x2D4ED32C [27] = 0xEDFC0000
[28] = 0xBFB0D5E1 [29] = 0xB50224B0 [30] = 0xC2603000 [31] = 0x00400300
qi@ISLA:~/COD25-Simulation-Framework$
```

```
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000

User: R??????????AU----------

User: RR 0 32;
00000000 A47AD480 7F665C66 00400550
00000000 004005A4 B50224B0 7053C7BA
004005F8 00000000 C14089D2 00400514
00000000 B2D9E7EC 8F4B55FE 03FB32E3
03FB32E3 E2D1098E EDFC0000 004005A4
00400848 00400550 70D502A5 004005F8
A47AD480 00400514 2D4ED32C EDFC0000
BFB0D5E1 B50224B0 C2603000 00400300

User:
```

四、总结

本次实验，实现了写优先的寄存器堆，并在此基础上对上次实验实现的单周期 CPU 进行修改，从而实现无冒险流水线 CPU，并进行两个层面的检验，从而验证了 CPU 的正确性。通过本次实验，对流水线有了进一步的了解，为后面实现完整流水线 CPU 打下基础。